

CHAPTER FIVE

Database Recovery Techniques

Databases Recovery

1. Recovery Concepts

I. Types of Failure

II. Transaction Log

III. Write ahead Log

IV. Data Updates

V. Data Caching

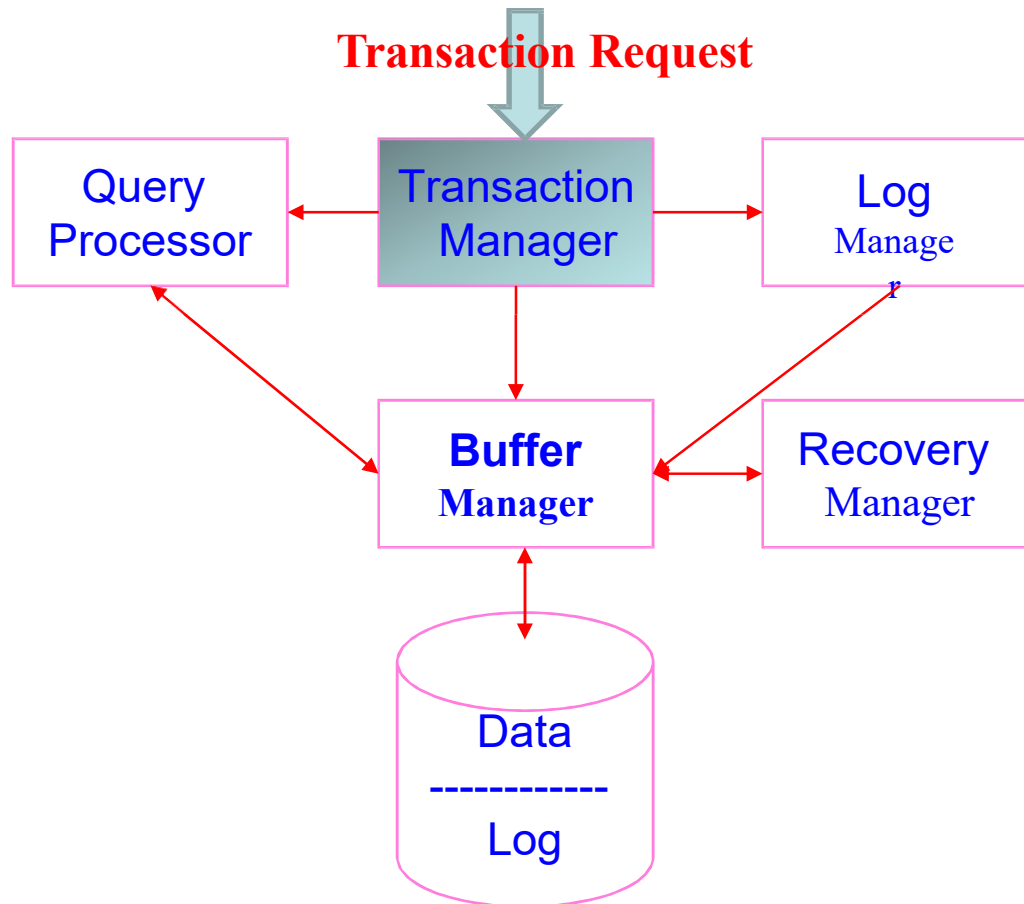
VI. Checkpointing

VII. Transaction Roll-back (Undo) and
Roll-Forward

VIII. Aries recovery algorithm

- **Transaction Manager** : Accepts transaction commands from an application, which tell the transaction manager when transactions begin and end, as well as information about the expectations of the application.

The transaction processor performs the following tasks:



✓ **Logging**: In order to assure *durability*, every change in the database is logged separately on disk.

✓ **Log manager** initially writes the log in buffers and negotiates with the buffer manager to make sure that buffers are written to disk at appropriate times.

✓ **Recovery Manager**: will be able to examine the log of changes and restore the database to some consistent state.

1. Recovery Concept

i. Purpose of Database Recovery

- To bring the database into the last consistent/ reliable state, which existed prior to the failure.
- To preserve/ safeguard transaction properties (**Atomicity & Durability**).
 - The recovery manager of a DBMS is responsible to ensure
 - **atomicity** by undoing the action of transaction that do not commit
 - **Durability** by making sure that all actions of committed transaction survive system crash

ii. Types of Failure

- The database may become unavailable for use due to:
 - **Transaction failure:** Transactions may fail because of incorrect input, deadlock.
 - **System failure:** System may fail because of addressing error, application error, operating system fault, RAM failure, etc.
 - **Media failure:** Disk head crash, power disruption, etc.

- To recover from system failure , the system keeps information about the change in the **system log**
- Strategy for recovery may be summarized as :
 - **Recovery from catastrophic**
 - If there is extensive damage to the wide portion of the database
 - This method restore a past copy of the database from the backup storage and reconstructs operation of a committed transaction from the back up log up to the time of failure
 - **Recovery from non-catastrophic failure**
 - When the database is *not physically damaged* but has become inconsistent
 - The strategy uses **undoing and redoing** some operations in order to restore to a consistent state.

■ For instance,

- If failure occurs between **commit** and database buffers being flushed to secondary storage then, to ensure durability, recovery manager has to *redo (roll forward)* transaction's updates.
- If transaction had **not committed** at failure time, recovery manager has to *undo (rollback)* any effects of that transaction for atomicity.

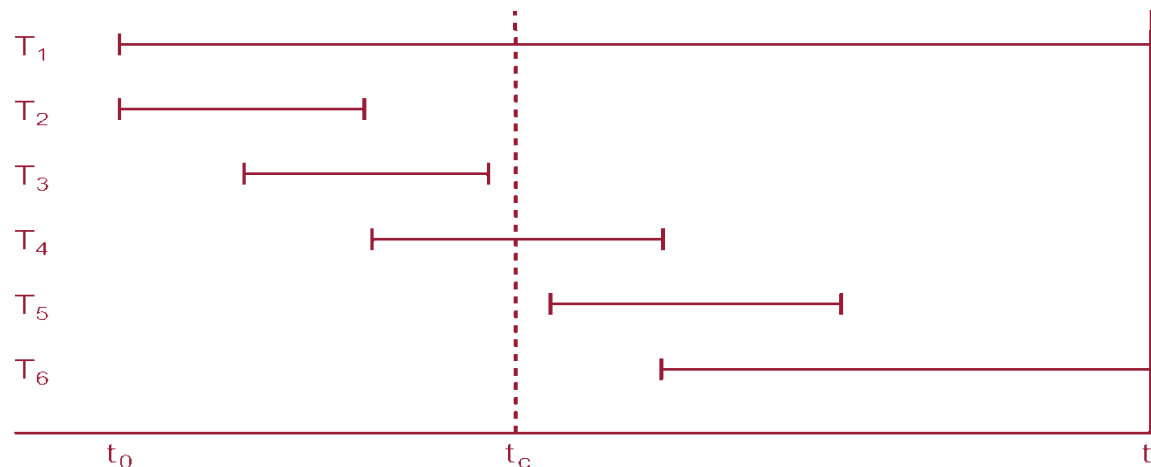


Fig 2 :
status of
transactions
at the time
of system
fails

- DBMS starts at time t_0 , but fails at time t_f .
- T_1 and T_6 have to be undone. In absence of any other information, recovery manager has to redo T_2 , T_3 , T_4 , and T_5 .

iii. Transaction Log

- For recovery from any type of failure data values prior to modification (BFIM - BeFore Image) and the new value after modification (AFIM – AAfter Image) are required.
- These values and other information is stored in a sequential file (appended file) called *Transaction log*
- These log files becomes very useful in brining back the system to a stable state after a system crash.
- A sample log is given below. Back P and Next P point to the previous and next log records of the same transaction.

T ID	Back P	Next P	Operation	Data item	BFIM	AFIM
T1	0	1	Begin			
T1	1	4	Write	X	X = 100	X = 200
T2	0	8	Begin			
T1	2	5	W	Y	Y = 50	Y = 100
T1	4	7	R	M	M = 200	M = 200
T3	0	9	R	N	N = 400	N = 400
T1	5	nil	End			

IV. Data Update : Four types

– Deferred Update:

- All transaction updates are recorded in the local workspace (cache)
- All modified data items in the cache is then written after a transaction ends its execution or after a fixed number of transactions have completed their execution.
- During commit the updates are first recorded on the log and then on the database
- If a transaction fails before reaching its commit point undo is not needed because it didn't change the database yet
- If a transaction fails after commit (writing on the log) but before finishing saving to the data base redoing is needed from the log

– Immediate Update:

- As soon as a data item is modified in cache, the disk copy is updated.
- These update are first recorded *on the log and on the disk* by force writing, before the database is updated
- If a transaction fails after recording some change to the database but before reaching its commit point , this will be rolled back

–Shadow update:

- The modified version of a data item does not overwrite its disk copy but is written at a separate disk location.
- Multiple version of the same data item can be maintained
- Thus the old value (before image BFIM) and the new value (AFIM) are kept in the disk
- No need of Log for recovery

–**In-place update:** The disk version of the data item is overwritten by the cache version.

V. Data Caching

- Data items to be modified are first stored into database cache by the Cache Manager (CM) and after modification they are flushed (written) to the disk
- When DBMS request for read /write operation on some item
 - It check the requested data item is in the cache or not
 - If it is not, the appropriate disk block are copied to the cache
 - If the cache is already full, some buffer replacement policy can be used . Like
 - Least recent used (LRU)
 - FIFO
 - While replacing buffers , first of all the updated value on that buffer should be saved on the appropriate block in the data base

VI. Write-Ahead Logging

- When *in-place* update (immediate or deferred) is used then log is necessary for recovery
- This log must be available to recovery manager
- This is achieved by *Write-Ahead Logging (WAL)* protocol. **WAL states that**
 - **For Undo:** Before a data item's AFIM is flushed to the database disk (overwriting the BFIM) its BFIM must be written to the log and the log must be saved on a stable store (log disk).
 - **For Redo:** Before a transaction executes its commit operation, all its AFIMs must be written to the log and the log must be saved on a stable store.

□ Standard Recovery Terminology

■ Possible ways for flushing database cache to database disk:

- i. **No-Steal**: Cache cannot be flushed before transaction commit.
- ii. **Steal**: Cache can be flushed before transaction commits.

Advantage:

- it avoids the need for a very large buffer space to store all updated pages in the memory

- iii. **Force**: if all Cache updates are immediately flushed (forced) to disk when a transaction commits-----→ force writing

- iv. **No-Force**: if Cached are flushed to a disk when the need arise after a committed transaction

Advantage:

- an updated pages of a committed transaction may still be in the buffer when an other transaction needs to update
- If this page is updated by multiple transaction, it eliminates the I/O cost to read that page again ,

■ These give rise to four different ways for handling recovery:

- Steal/No-Force (Undo/Redo)
- Steal/Force (Undo/No-redo)
- No-Steal/No-Force (No-undo/Redo)
- No-Steal/Force (No-undo/No-redo)

VII. Checkpointing

- Log file is used to recover failed DB but we may not know how far back in the log to search. Thus
- Checkpoint is a Point of synchronization between database and log file.
- Time to time (randomly or under some criteria) the database flushes its buffer to database disk to minimize the task of recovery.
- When failure occurs, redo all transactions that committed since the checkpoint and undo all transactions active at time of crash.
- In previous example ,**Figure 2, on slide 6** , with checkpoint at time t_c , changes made by T2 and T3 have been written to secondary storage. Thus: *only redo T4 and T5, undo transactions T1 and T6.*
- At what interval to take a check pointing may be measured :
 - In terms of time , say m minutes after the last check point
 - The # t of committed transaction since the last check point
- The following steps defines a checkpoint operation:
 - i. Suspend execution of transactions temporarily.
 - ii. Force write modified buffer data to disk.
 - iii. Write a [checkpoint] record to the log, save the log to disk.
 - iii. Resume normal transaction execution.

Example

(a)

T_1	T_2	T_3
read_item(A)	read_item(B)	read_item(C)
read_item(D)	write_item(B)	write_item(B)
write_item(D)	read_item(D)	read_item(A)
	write_item(D)	write_item(A)

(a) The read and write operations of three transactions.

(b) System log at point of crash. (c) Operations before the crash.

(b)

	A	B	C	D
	30	15	40	20
	[start_transaction, T_3]			
	[read_item, T_3 , C]			
*	[write_item, T_3 , B, 15, 12]	12		
	[start_transaction, T_2]			
	[read_item, T_2 , B]			
**	[write_item, T_2 , B, 12, 18]	18		
	[start_transaction, T_1]			
	[read_item, T_1 , A]			
	[read_item, T_1 , D]			
	[write_item, T_1 , D, 20, 25]			25
	[read_item, T_2 , D]			
**	[write_item, T_2 , D, 25, 26]			26
	[read_item, T_3 , A]			

← System crash

(a) The read and write operations of three transactions.

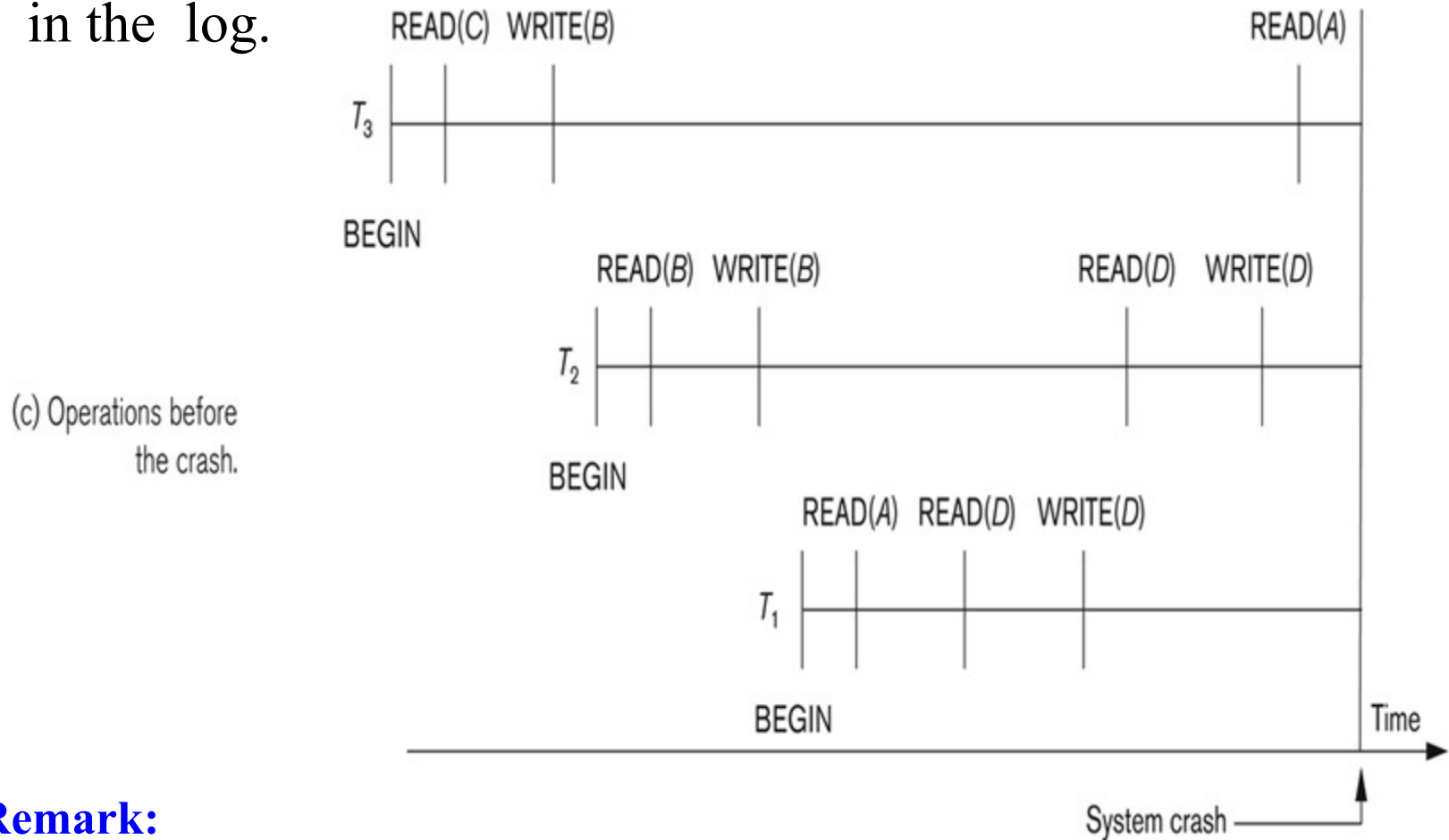
(b) System log at point of crash. (c) Operations before the crash.

* T_3 is rolled back because it did not reach its commit point.

** T_2 is rolled back because it reads the value of item B written by T_3 .

Roll-back:
in the log.

One execution of T1, T2 and T3 as recorded



Remark:

- Only write item operation need to be undone during transaction roll back
- Read item operations are recorded in the log only to determine whether cascading roll back of additional transaction is needed

2. Recovery Scheme

i. Deferred Update (No Undo/Redo)

- The data update goes as follows:
 - A set of transactions records their updates in the log.
 - At commit point under **WAL** scheme these updates are saved on database disk.
 - **No undo** is required because no AFIM is flushed to the disk before a transaction commits.
 - After reboot from a failure the log is used to **redo** all the transactions affected by this failure.
- **Limitation:** *out of buffer space may be happened because transaction changes must be held in the cache buffer until the commit point*
- Type of deferred updated recovery environment
 - **Single User** and **Multiple user environment**

a) Deferred Update in a single-user system

- There is no concurrent data sharing in a single user system.
- The data update goes as follows:
 - A set of transactions records their updates in the log.
 - At commit point under WAL scheme these updates are saved on database disk.

(a)

T_1	T_2
read_item(A)	read_item(B)
read_item(D)	write_item(B)
write_item(D)	read_item(D)
	write_item(D)

(b)

[start_transaction, T_1]
[write_item, $T_1, D, 20$]
[commit, T_1]
[start_transaction, T_2]
[write_item, $T_2, B, 10$]
[write_item, $T_2, D, 25$]

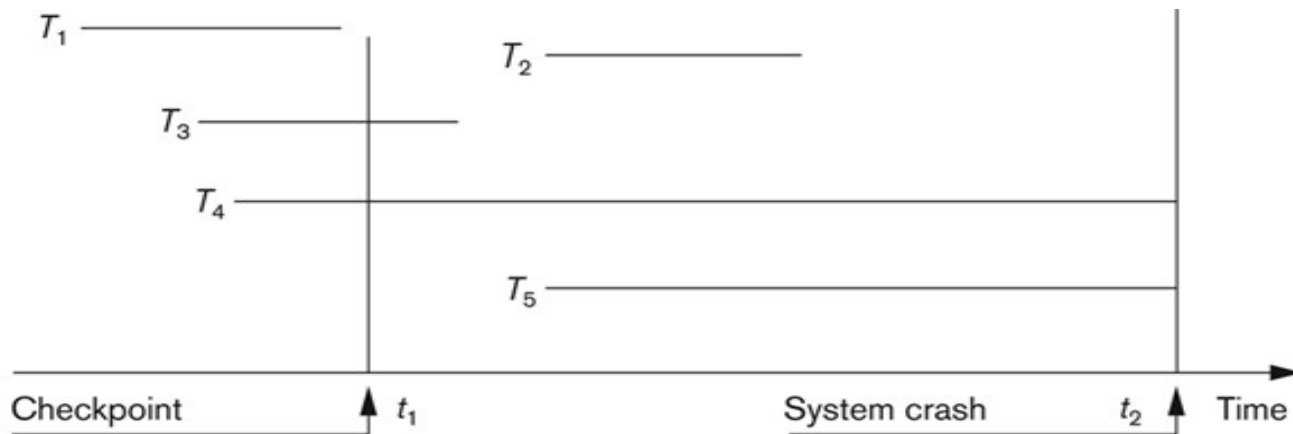
← System crash

An example of recovery using deferred update in a single-user environment. (a) The READ and WRITE operations of two transactions. (b) The system log at the point of crash.

The [write_item,...] operations of T_1 are redone.
 T_2 log entries are ignored by the recovery process.

b) Deferred Update with concurrent users

- This environment requires some concurrency control mechanism to guarantee **isolation** property of transactions.
- In a system recovery, transactions which were recorded in the log after the last checkpoint were **redone**.
- The recovery manager may scan some of the transactions recorded before the checkpoint to get the AFIMs.



An example of recovery in a multi-user environment

- T_4 and T_5 are ignored because they didn't reach their commit points
- T_2 and T_3 are redone because their commit point is after the last checkpoint

- Two tables are required for implementing this protocol:
 - **Active table:** All active transactions are entered in this table.
 - **Commit table:** Transactions to be committed are entered in this table.
- During recovery, all transactions of the **commit** table are redone and all transactions of **active** tables are ignored since none of their AFIMs reached the database.
- It is possible that a **commit** table transaction may be **redone** twice but this does not create any inconsistency because of a redone is “**idempotent**”,
- that is, one redone for an AFIM is equivalent to multiple redone for the same AFIM.

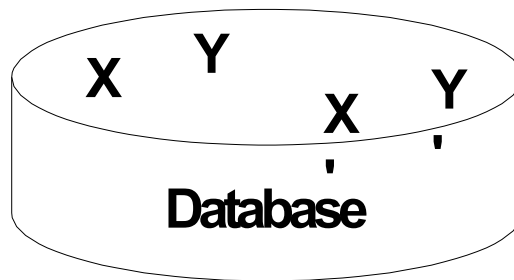
ii. Recovery Techniques Based on Immediate Update

■ Undo/No-redo Algorithm

- In this algorithm AFIMs of a transaction are flushed to the database disk under WAL before it commits.
- For this reason the recovery manager **undoes** all transactions during recovery.
- No transaction is **redone**.
- It is possible that a transaction might have completed execution and ready to commit but this transaction is also **undone**.

iii. Shadow Paging

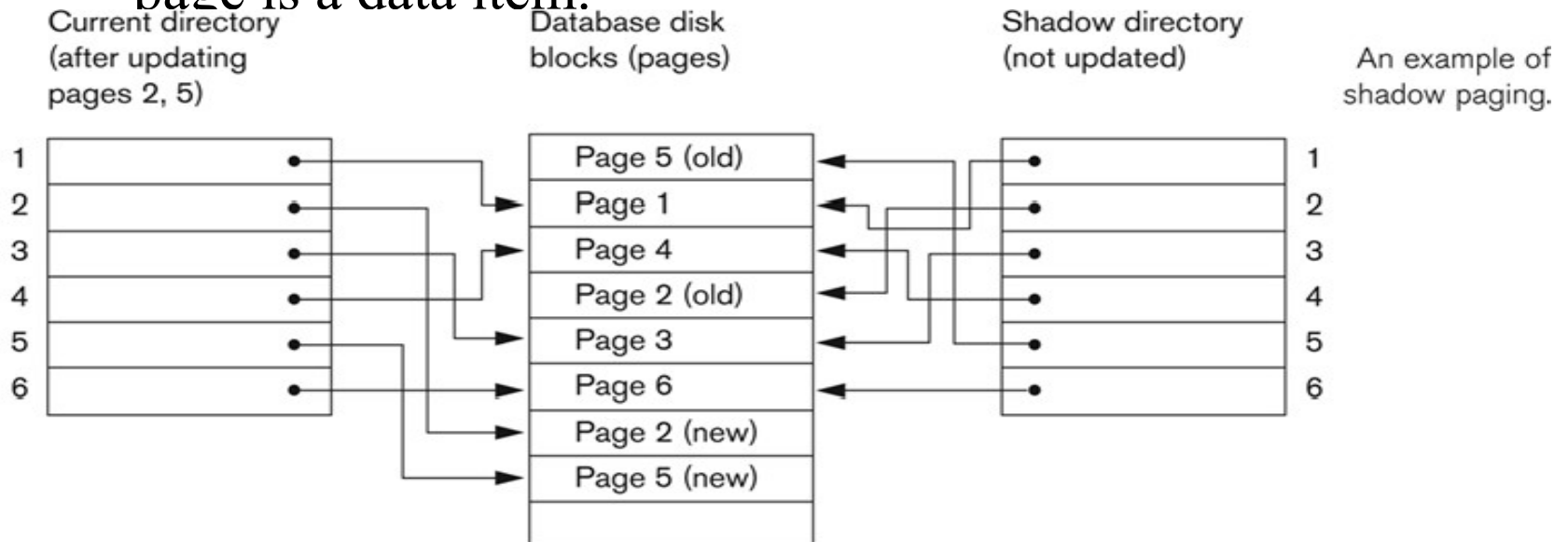
- Maintain two page tables during life of a transaction: *current* page and *shadow* page table.
- When transaction starts, two pages are the same.
- Shadow page table is never changed thereafter and is used to restore database in event of failure.
- During transaction, current page table records all updates to database.
- When transaction completes, current page table becomes shadow page table.



X and Y: Shadow copies of data items

X' and Y': Current copies of data items

- To manage access of data items by concurrent transactions two directories (current and shadow) are used.
 - The directory arrangement is illustrated below. Here a page is a data item.



- **Advantages** over Log Based :
 - Overhead of maintaining log is removed
 - Recovering is faster since there is no need for undo and redo
- **Disadvantages:**
 - Updated pages will change its location on the disk
 - Need Garbage collection after a transaction committed so as to free the page for future use
 - Migration between the current and shadow directories may not be

ARIES Recovery Algorithm

- ARIES stands for “**Algorithm for Recovery and Isolation Exploiting Semantics.**”
- ARIES is a state of the art recovery method
 - Incorporates numerous optimizations to reduce overheads during normal processing and to speed up recovery
 - The recovery algorithm we studied earlier is modeled after ARIES, but greatly simplified by removing optimizations
- Unlike the recovery algorithm described earlier, ARIES
 1. Uses **log sequence number (LSN)** to identify log records
 - ▶ Stores LSNs in pages to identify what updates have already been applied to a database page
 2. Physiological redo
 3. Dirty page table to avoid unnecessary redos during recovery
 4. Fuzzy checkpointing that only records information about dirty pages, and does not require dirty pages to be written out at checkpoint time

Cont.... Log Record

- ❑ Each log record contains LSN of previous log record of the same transaction
- ❑ LSN in log record may be implicit
- Special redo-only log record called **compensation log record (CLR)** used to log actions taken during recovery that never need to be undone
 - Serves the role of operation-abort log records used in earlier recovery algorithm
 - Has a field UndoNextLSN to note next (earlier) record to be undone
 - ▶ Records in between would have already been undone
 - ▶ Required to avoid repeated undo of already undone actions

Cont..

■ DirtyPageTable

- List of pages in the buffer that have been updated
- Contains, for each such page
 - ▶ **PageLSN** of the page
 - ▶ **RecLSN** is an LSN such that log records before this LSN have already been applied to the page version on disk
 - Set to current end of log when a page is inserted into dirty page table (just before being updated)
 - Recorded in checkpoints, helps to minimize redo work

Cont...

■ Checkpoint log record

- Contains:
 - ▶ DirtyPageTable and list of active transactions
 - ▶ For each active transaction, LastLSN, the LSN of the last log record written by the transaction
- Fixed position on disk notes LSN of last completed checkpoint log record

- Dirty pages are not written out at checkpoint time
 - ▶ Instead, they are flushed out continuously, in the background

- Checkpoint is thus very low overhead
 - can be done frequently

Cont...

➤ **The recovery process actually consists of 3 phases:**

1.Analysis: This phase reads the last checkpoint record in the log to figure out active transactions and dirty pages at point of crash/restart. A page is considered dirty if it was modified in memory but was not written to disk. This information is used by next two phases.

2.Redo: Starting at the earliest LSN, the log is read forward and each update redone.

3.Undo: The log is scanned backward and updates corresponding to loser transactions are undone.

The end !!!!
Thank you!!!!!!